

```

#include <stdio.h>
#include <string.h>

#define MAX_EMPLOYEES 100

struct Employee
{
    int id;
    char name[50];
    char position[50];
    float salary;
    char joinDate[20];
    int isActive;
    char leaveDate[20];
};

struct Employee employees[MAX_EMPLOYEES];
int employeeCount = 0;

void addEmployee();
void displayEmployees();
void searchEmployee();
void updateEmployee();
void deleteEmployee();
void sortEmployees();
void saveEmployees();
void loadEmployees();
void printCenter();

int main()
{
    int choice;

    loadEmployees();

    do
    {
        printf("\n===== EMPLOYEE RECORD SYSTEM =====\n");
        printf("1. Add Employee\n");
        printf("2. Display All Employees\n");
        printf("3. Search Employee\n");
        printf("4. Update Employee\n");
        printf("5. Delete Employee\n");
        printf("6. Sort Employees\n");
        printf("7. Save to File\n");
        printf("8. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                addEmployee();
                break;

            case 2:
                displayEmployees();
                break;

            case 3:
                searchEmployee();
                break;

            case 4:

```

```

        updateEmployee();
        break;

    case 5:
        deleteEmployee();
        break;

    case 6:
        sortEmployees();
        break;

    case 7:
        saveEmployees();
        break;

    case 8:
        saveEmployees();
        printf("Exiting program. Goodbye!\n");
        break;

    default:
        printf("Invalid choice.\n");
    }

    } while (choice != 8);

    return 0;
}

// Add employee
void addEmployee()
{
    struct Employee e;
    int i;

    if (employeeCount >= MAX_EMPLOYEES)
    {
        printf("Employee list is full.\n");
        return;
    }

    printf("Enter Employee ID: ");
    scanf("%d", &e.id);

    for (i = 0; i < employeeCount; i++)
    {
        if (employees[i].id == e.id)
        {
            printf("This ID already exists.\n");
            return;
        }
    }

    printf("Enter Name: ");
    scanf("%[^\n]", e.name);

    printf("Enter Position: ");
    scanf("%[^\n]", e.position);

    printf("Enter Salary: ");
    scanf("%f", &e.salary);

    printf("Enter Join Date (DD-MM-YYYY): ");

```

```

scanf("%19s", e.joinDate);

printf("Is the employee active? (1 = Yes, 0 = No): ");
scanf("%d", &e.isActive);

if (e.isActive == 1)
{
    strcpy(e.leaveDate, "Present");
}
else
{
    printf("Enter Leave Date (DD-MM-YYYY): ");
    scanf("%19s", e.leaveDate);
}

employees[employeeCount] = e;
employeeCount++;

printf("Employee added successfully.\n");
}

// Display all employees
void displayEmployees()
{
    int i;
    int id;
    int choice;

    int nameWidth = 4;
    int positionWidth = 8;
    int salaryWidth = 8;
    int statusWidth = 10;
    int joinDateWidth = 9;
    int leaveDateWidth = 10;

    int totalWidth;

    if (employeeCount == 0)
    {
        printf("No employees found.\n");
        return;
    }

    /* Find longest name */
    for (i = 0; i < employeeCount; i++)
    {
        if (strlen(employees[i].name) > nameWidth)
        {
            nameWidth = strlen(employees[i].name);
        }
    }

    /* Find longest position */
    for (i = 0; i < employeeCount; i++)
    {
        if (strlen(employees[i].position) > positionWidth)
        {
            positionWidth = strlen(employees[i].position);
        }
    }

    /* Find longest join date */
    for (i = 0; i < employeeCount; i++)

```

```

    {
        if (strlen(employees[i].joinDate) > joinDateWidth)
        {
            joinDateWidth = strlen(employees[i].joinDate);
        }
    }

    /* Find longest leave date */
    for (i = 0; i < employeeCount; i++)
    {
        if (strlen(employees[i].leaveDate) > leaveDateWidth)
        {
            leaveDateWidth = strlen(employees[i].leaveDate);
        }
    }

    /*
        Salary needs enough space for numbers such as:
        50000.00
    */

    if (salaryWidth < 8)
    {
        salaryWidth = 8;
    }

    /*
        Total table width
        7 columns
        8 vertical lines
        2 spaces around each column
    */

    totalWidth = 5 + nameWidth + positionWidth
                + salaryWidth + statusWidth
                + joinDateWidth + leaveDateWidth
                + 14 + 8;

    printf("\n");

    /* Top border */

    for (i = 0; i < totalWidth; i++)
    {
        printf("-");
    }

    printf("\n");

    /* Table header */

    printf("| ");
    printCenter("ID", 3);

    printf(" | ");
    printCenter("Name", nameWidth);

    printf(" | ");
    printCenter("Position", positionWidth);

    printf(" | ");
    printCenter("Salary", salaryWidth);

```

```

printf(" | ");
printCenter("Status", statusWidth);

printf(" | ");
printCenter("Join Date", joinDateWidth);

printf(" | ");
printCenter("Leave Date", leaveDateWidth);

printf(" |\n");

/* Header separator */

for (i = 0; i < totalWidth; i++)
{
    printf("-");
}

printf("\n");

/* Employee rows */

for (i = 0; i < employeeCount; i++)
{
    printf("| %-3d ", employees[i].id);

    printf("| %-*s ", nameWidth, employees[i].name);

    printf("| %-*s ", positionWidth, employees[i].position);

    printf("| %*.2f ", salaryWidth, employees[i].salary);

    if (employees[i].isActive == 1)
    {
        printf("| %-*s ", statusWidth, "Active");
    }
    else
    {
        printf("| %-*s ", statusWidth, "Not Active");
    }

    printf("| %-*s ", joinDateWidth, employees[i].joinDate);

    printf("| %-*s ", leaveDateWidth, employees[i].leaveDate);

    printf("|\n");
}

/* Bottom border */

for (i = 0; i < totalWidth; i++)
{
    printf("-");
}

printf("\n");

/* Individual search */

```

```

printf("\nDo you want to search for an individual employee?\n");
printf("1. Yes\n");
printf("2. No\n");
printf("Enter your choice: ");
scanf("%d", &choice);

if (choice == 1)
{
    printf("Enter Employee ID: ");
    scanf("%d", &id);

    for (i = 0; i < employeeCount; i++)
    {
        if (employees[i].id == id)
        {
            printf("\nEmployee Details\n");
            printf("-----\n");

            printf("ID: %d\n", employees[i].id);
            printf("Name: %s\n", employees[i].name);
            printf("Position: %s\n", employees[i].position);
            printf("Salary: %.2f\n", employees[i].salary);
            printf("Join Date: %s\n", employees[i].joinDate);

            if (employees[i].isActive == 1)
            {
                printf("Status: Active\n");
            }
            else
            {
                printf("Status: Not Active\n");
            }

            printf("Leave Date: %s\n", employees[i].leaveDate);

            return;
        }
    }

    printf("Employee not found.\n");
}

}

// Center align table heading
void printCenter(char text[], int width)
{
    int length;
    int leftSpace;
    int rightSpace;
    int i;

    length = strlen(text);

    if (length >= width)
    {
        printf("%s", text);
        return;
    }

    leftSpace = (width - length) / 2;
    rightSpace = width - length - leftSpace;

    for (i = 0; i < leftSpace; i++)

```

```

        {
            printf(" ");
        }

        printf("%s", text);

        for (i = 0; i < rightSpace; i++)
        {
            printf(" ");
        }
    }

// Search employee by ID
void searchEmployee()
{
    int id;
    int i;
    int found = 0;

    printf("Enter Employee ID to search: ");
    scanf("%d", &id);

    for (i = 0; i < employeeCount; i++)
    {
        if (employees[i].id == id)
        {
            printf("\nEmployee Details\n");
            printf("-----\n");

            printf("ID: %d\n", employees[i].id);
            printf("Name: %s\n", employees[i].name);
            printf("Position: %s\n", employees[i].position);
            printf("Salary: %.2f\n", employees[i].salary);
            printf("Join Date: %s\n", employees[i].joinDate);

            if (employees[i].isActive == 1)
            {
                printf("Status: Active\n");
            }
            else
            {
                printf("Status: Not Active\n");
            }

            printf("Leave Date: %s\n", employees[i].leaveDate);

            found = 1;
            break;
        }
    }

    if (found == 0)
    {
        printf("Employee not found.\n");
    }
}

// Update employee
void updateEmployee()
{
    int id;
    int i;

```

```

int choice;

printf("Enter Employee ID to update: ");
scanf("%d", &id);

for (i = 0; i < employeeCount; i++)
{
    if (employees[i].id == id)
    {
        printf("\nWhat do you want to update?\n");
        printf("1. Name\n");
        printf("2. Position\n");
        printf("3. Salary\n");
        printf("4. Join Date\n");
        printf("5. Active Status\n");
        printf("6. Leave Date\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                printf("Enter new Name: ");
                scanf(" %[^\\n]", employees[i].name);
                break;

            case 2:
                printf("Enter new Position: ");
                scanf(" %[^\\n]", employees[i].position);
                break;

            case 3:
                printf("Enter new Salary: ");
                scanf("%f", &employees[i].salary);
                break;

            case 4:
                printf("Enter new Join Date (DD-MM-YYYY): ");
                scanf("%19s", employees[i].joinDate);
                break;

            case 5:
                printf("Is the employee active? (1 = Yes, 0 = No): ");
                scanf("%d", &employees[i].isActive);

                if (employees[i].isActive == 1)
                {
                    strcpy(employees[i].leaveDate, "Present");
                }
                else
                {
                    printf("Enter Leave Date (DD-MM-YYYY): ");
                    scanf("%19s", employees[i].leaveDate);
                }
                break;

            case 6:
                printf("Enter new Leave Date (DD-MM-YYYY): ");
                scanf("%19s", employees[i].leaveDate);

                employees[i].isActive = 0;
                break;

            default:

```

```

        printf("Invalid choice.\n");
        return;
    }

    printf("Employee information updated successfully.\n");
    return;
}

printf("Employee not found.\n");
}

// Delete employee
void deleteEmployee()
{
    int id;
    int i;
    int j;
    int position = -1;
    int confirm;

    printf("Enter Employee ID to delete: ");
    scanf("%d", &id);

    for (i = 0; i < employeeCount; i++)
    {
        if (employees[i].id == id)
        {
            position = i;
            break;
        }
    }

    if (position == -1)
    {
        printf("Employee not found.\n");
        return;
    }

    printf("\nEmployee Details\n");
    printf("-----\n");

    printf("ID: %d\n", employees[position].id);
    printf("Name: %s\n", employees[position].name);
    printf("Position: %s\n", employees[position].position);

    printf("\nAre you sure you want to delete this employee? (1 = Yes, 0 = No): ");
    scanf("%d", &confirm);

    if (confirm == 1)
    {
        for (j = position; j < employeeCount - 1; j++)
        {
            employees[j] = employees[j + 1];
        }

        employeeCount--;

        printf("Employee deleted successfully.\n");
    }
    else
    {

```

```

        printf("Delete cancelled.\n");
    }
}

// Sort employees using Bubble Sort
void sortEmployees()
{
    int i;
    int j;
    int choice;
    struct Employee temp;

    if (employeeCount == 0)
    {
        printf("No employees to sort.\n");
        return;
    }

    printf("\nSort by:\n");
    printf("1. ID\n");
    printf("2. Salary\n");
    printf("Enter choice: ");
    scanf("%d", &choice);

    if (choice == 1)
    {
        for (i = 0; i < employeeCount - 1; i++)
        {
            for (j = 0; j < employeeCount - i - 1; j++)
            {
                if (employees[j].id > employees[j + 1].id)
                {
                    temp = employees[j];
                    employees[j] = employees[j + 1];
                    employees[j + 1] = temp;
                }
            }
        }

        printf("Employees sorted by ID.\n");
    }
    else if (choice == 2)
    {
        for (i = 0; i < employeeCount - 1; i++)
        {
            for (j = 0; j < employeeCount - i - 1; j++)
            {
                if (employees[j].salary > employees[j + 1].salary)
                {
                    temp = employees[j];
                    employees[j] = employees[j + 1];
                    employees[j + 1] = temp;
                }
            }
        }

        printf("Employees sorted by Salary.\n");
    }
    else
    {
        printf("Invalid choice.\n");
    }
}

```

```

// Save employees to file
void saveEmployees()
{
    FILE *file;
    int i;

    file = fopen("employees.txt", "w");

    if (file == NULL)
    {
        printf("File could not be opened.\n");
        return;
    }

    fprintf(file, "%d\n", employeeCount);

    for (i = 0; i < employeeCount; i++)
    {
        fprintf(file, "%d\n", employees[i].id);
        fprintf(file, "%s\n", employees[i].name);
        fprintf(file, "%s\n", employees[i].position);
        fprintf(file, "%.2f\n", employees[i].salary);
        fprintf(file, "%s\n", employees[i].joinDate);
        fprintf(file, "%d\n", employees[i].isActive);
        fprintf(file, "%s\n", employees[i].leaveDate);
    }

    fclose(file);

    printf("Saved to file.\n");
}

// Load employees from file
void loadEmployees()
{
    FILE *file;
    int i;

    file = fopen("employees.txt", "r");

    if (file == NULL)
    {
        return;
    }

    fscanf(file, "%d\n", &employeeCount);

    if (employeeCount > MAX_EMPLOYEES)
    {
        employeeCount = MAX_EMPLOYEES;
    }

    for (i = 0; i < employeeCount; i++)
    {
        fscanf(file, "%d\n", &employees[i].id);

        fgets(employees[i].name, 50, file);
        employees[i].name[strcspn(employees[i].name, "\n")] = '\0';

        fgets(employees[i].position, 50, file);
        employees[i].position[strcspn(employees[i].position, "\n")] = '\0';
    }
}

```

```
        fscanf(file, "%f\n", &employees[i].salary);
        fscanf(file, "%19s\n", employees[i].joinDate);
        fscanf(file, "%d\n", &employees[i].isActive);
        fscanf(file, "%19s\n", employees[i].leaveDate);
    }

    fclose(file);
}
```